CS-229

Dynamics week!

- Adam and SGD

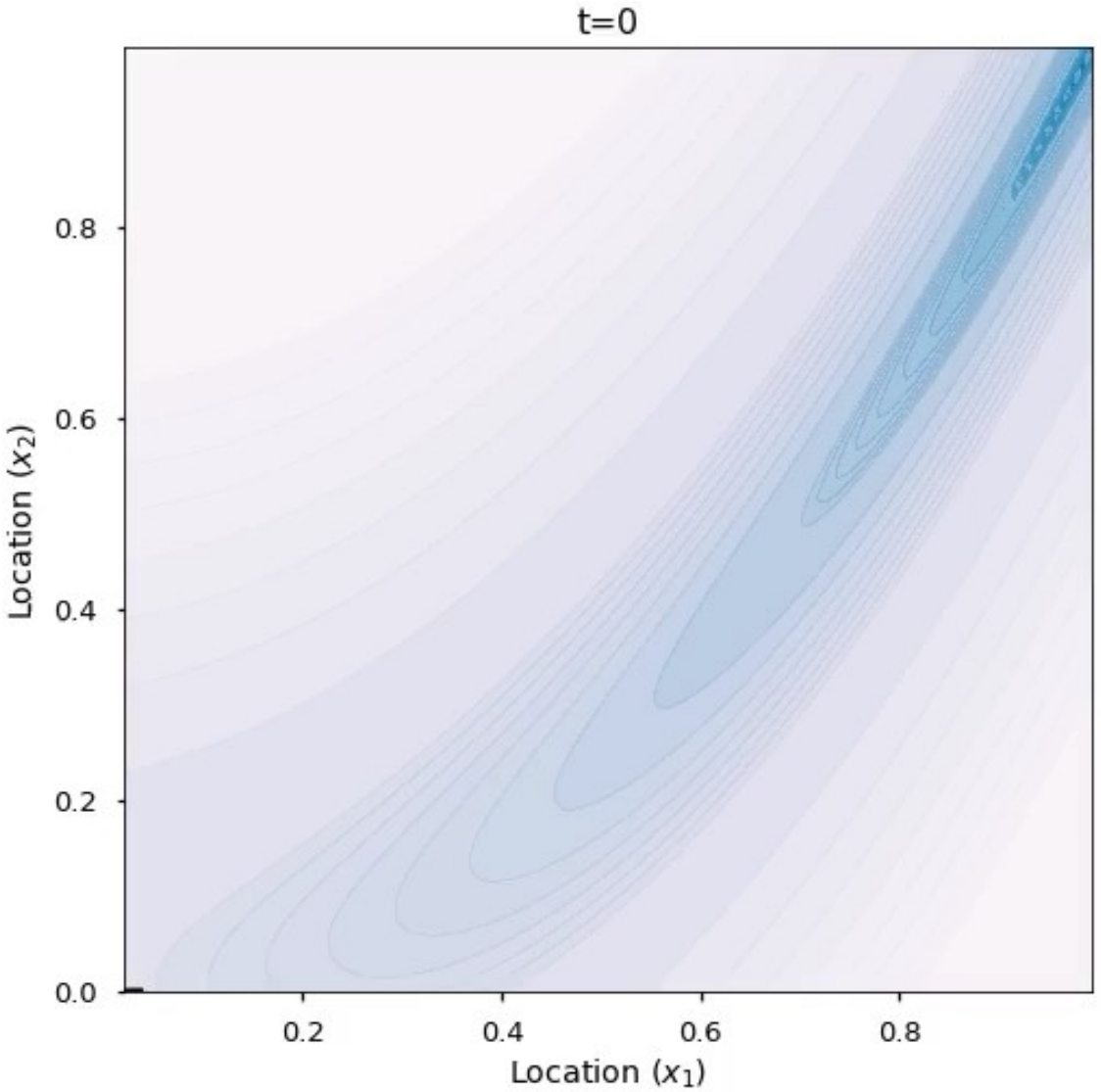
Next week:

Go deep on unsupervised learning

variational methods and diffusion

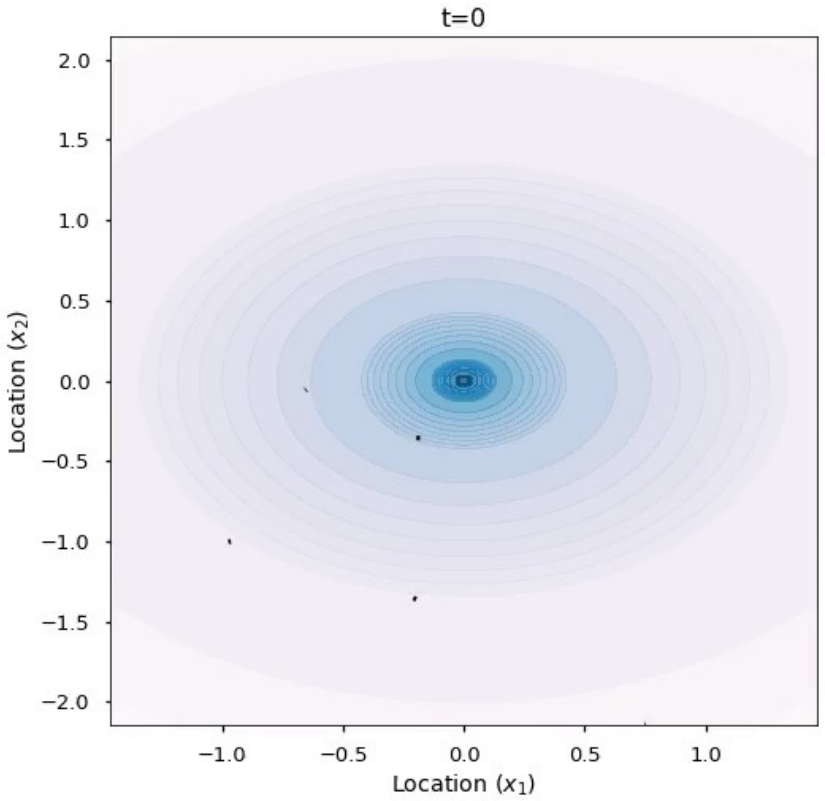
Quick recap of Monday

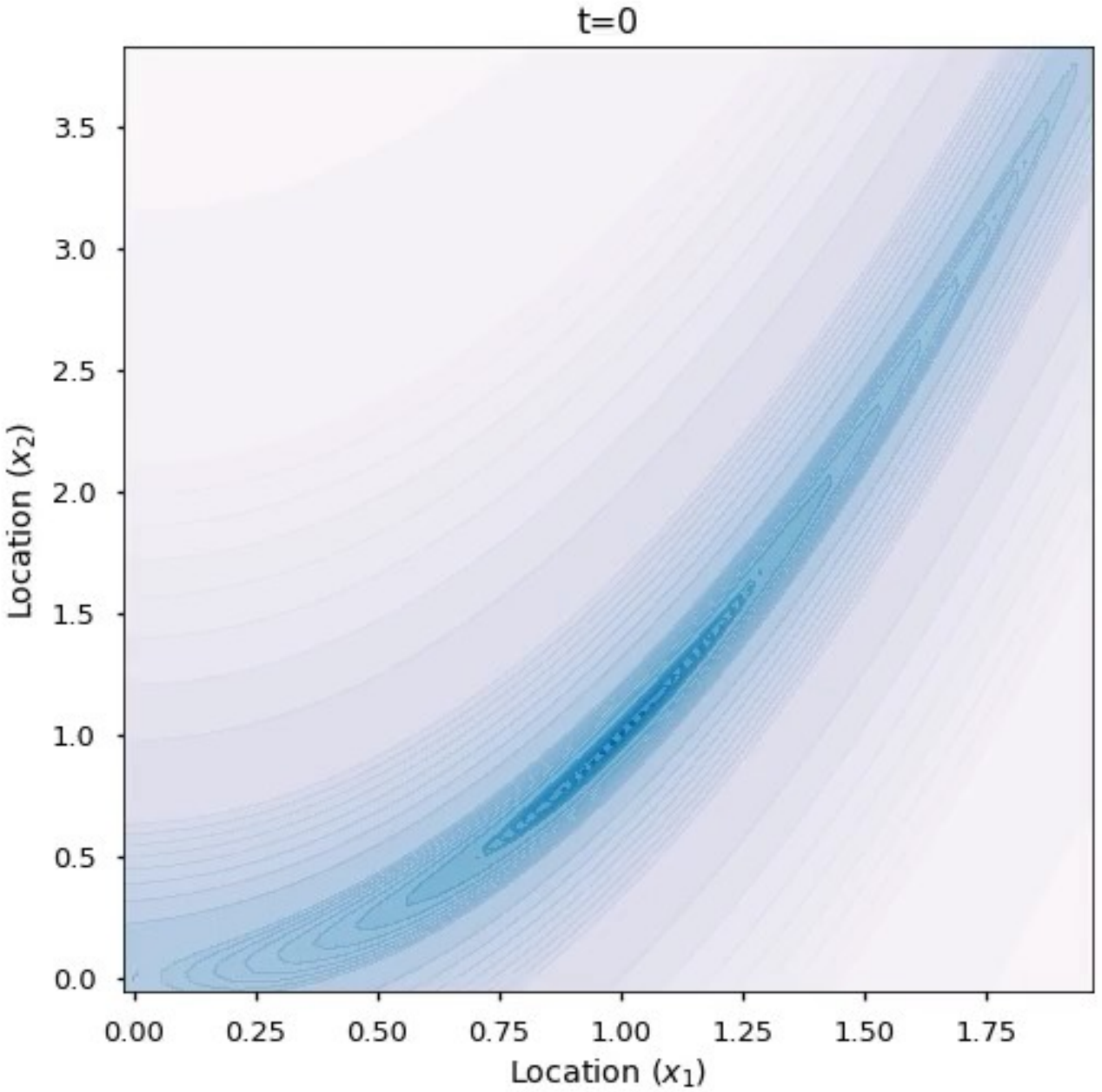
Classic “one dot” slow down (gradient descent)

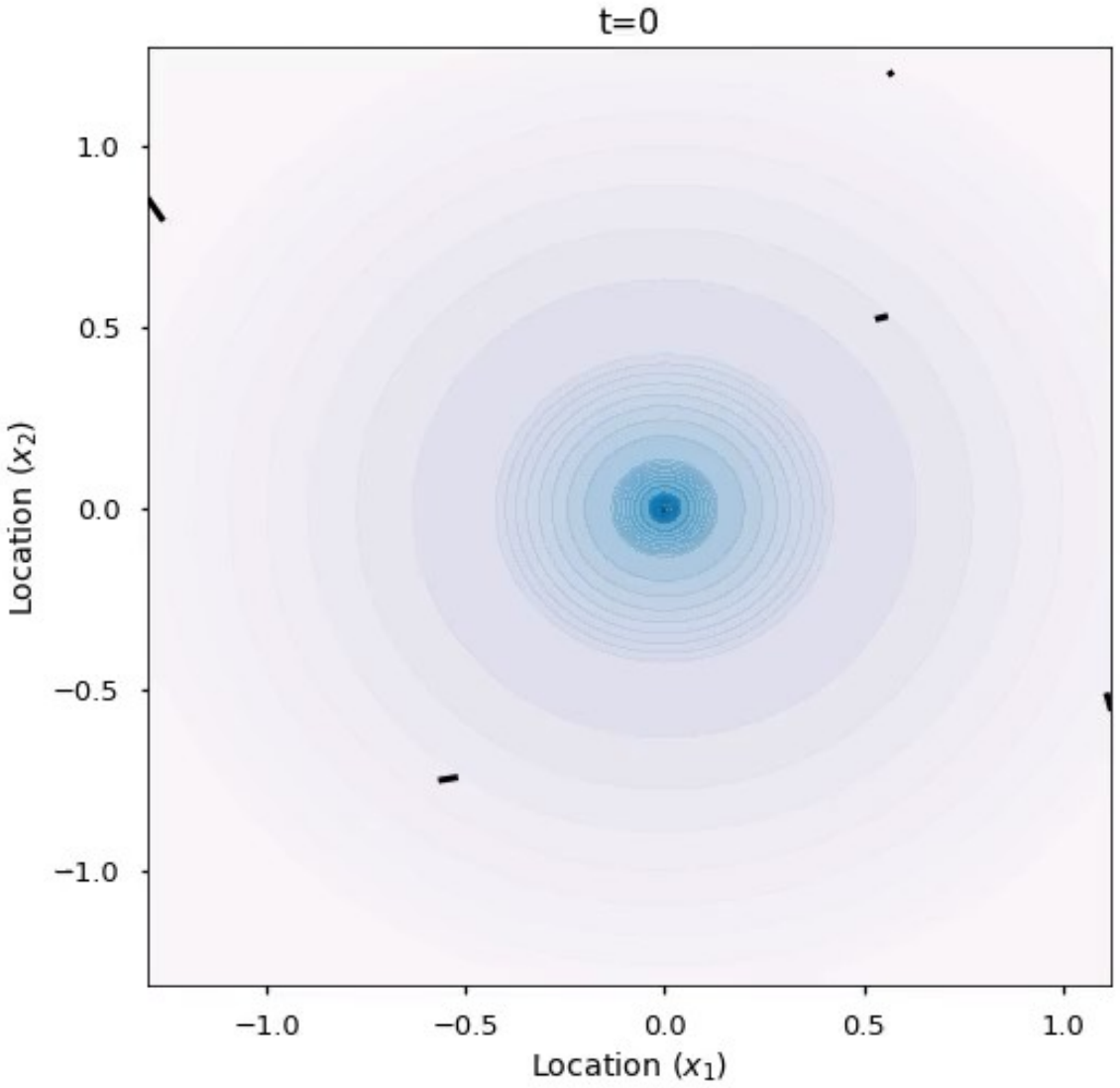


The difference has to do with *momentum*

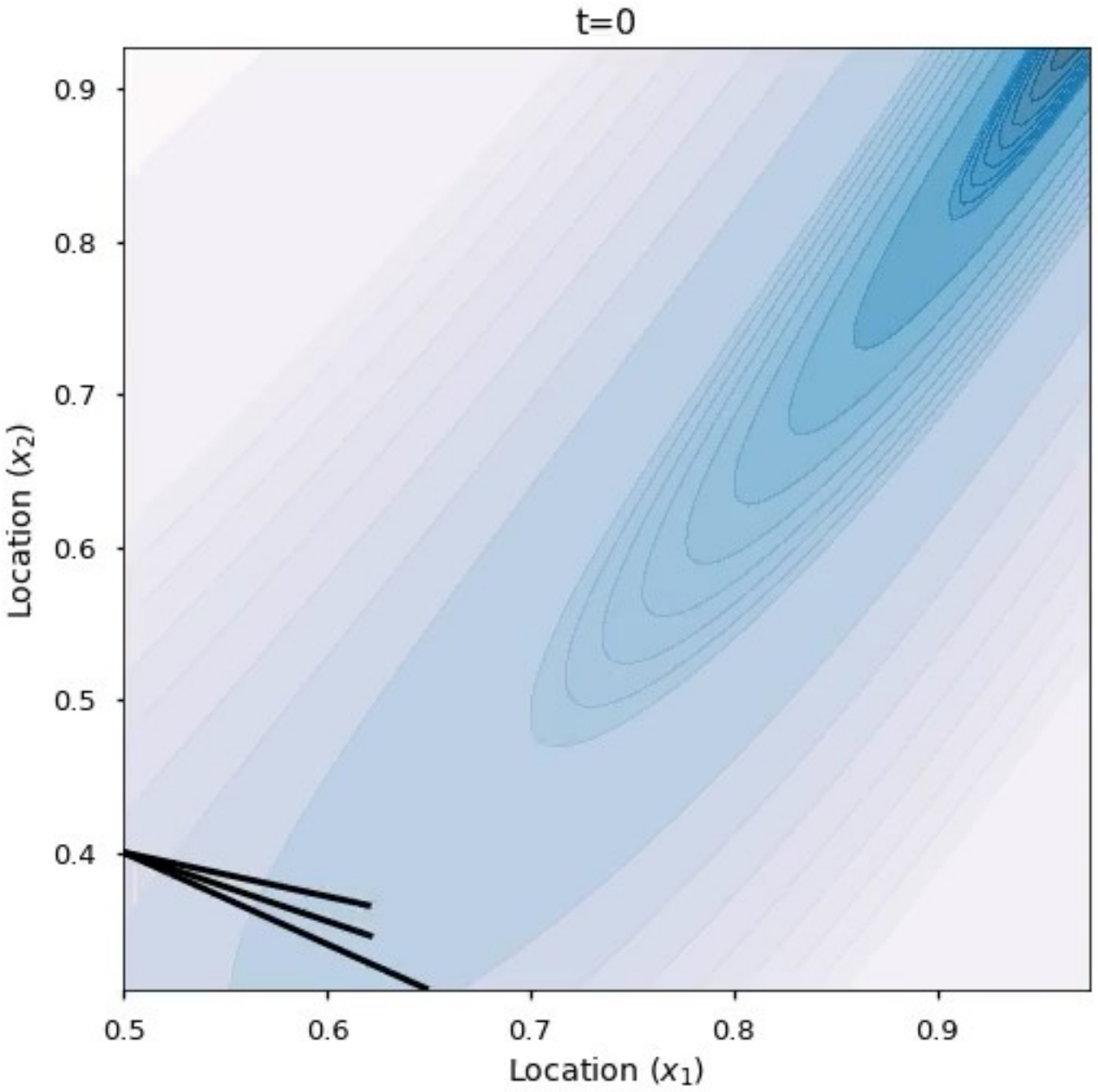
Hamiltonian dynamics (momentum but no friction)







- Hamiltonian dynamics with friction
- Classic (static) friction slows things down too much!
- (Hence default is “nesterov” momentum)



Using large step size is implicitly like adding acceleration, but can lead to numerical errors

Chaos was hiding in plain sight all along!

Chaos in training dynamics?

https://arxiv.org/pdf/2604.19740

(Also maybe solves the mystery of limit cycles paper)

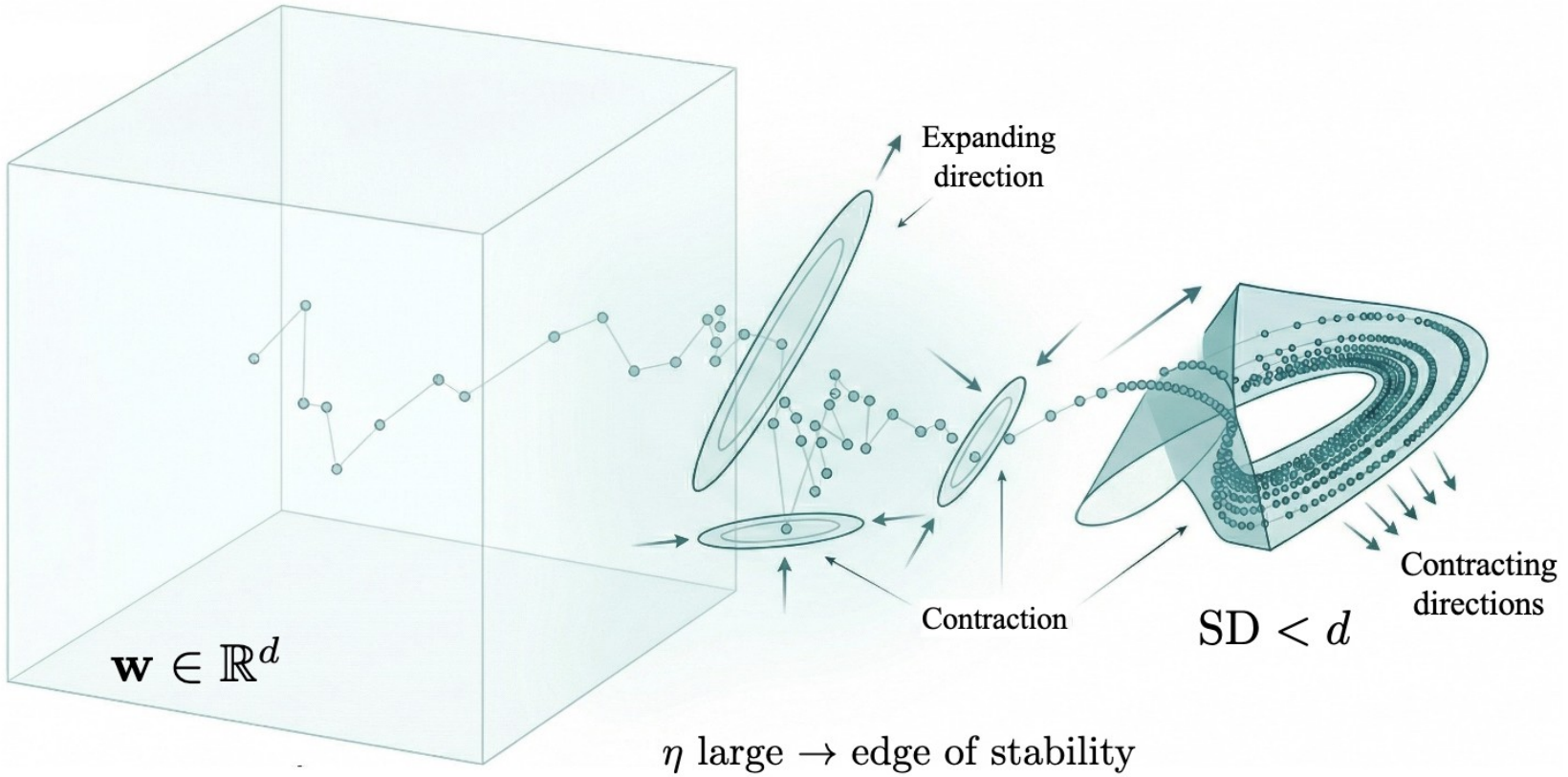


Figure 1: **Generalization at the Edge of Stability (EoS).** Modeling stochastic optimization as a random dynamical system (RDS), we show that at EoS the leading sharpness satisfies $\lambda_1 > 0$, implying expansion along at least one direction. The fundamental balance between expansion and contraction implies that the effective dimensionality of the dynamics, measured by our **Sharpness Dimension** (SD), is strictly smaller than the ambient parameter space: SD $< d$. We prove that the **worst-case generalization error is governed by** SD **rather than the parameter count**. Our results identify EoS as precisely the regime where generalization is controlled by a provably lower-dimensional attractor, providing a principled explanation for why overparameterized models can generalize beyond classical complexity measures.

From momentum to Adam

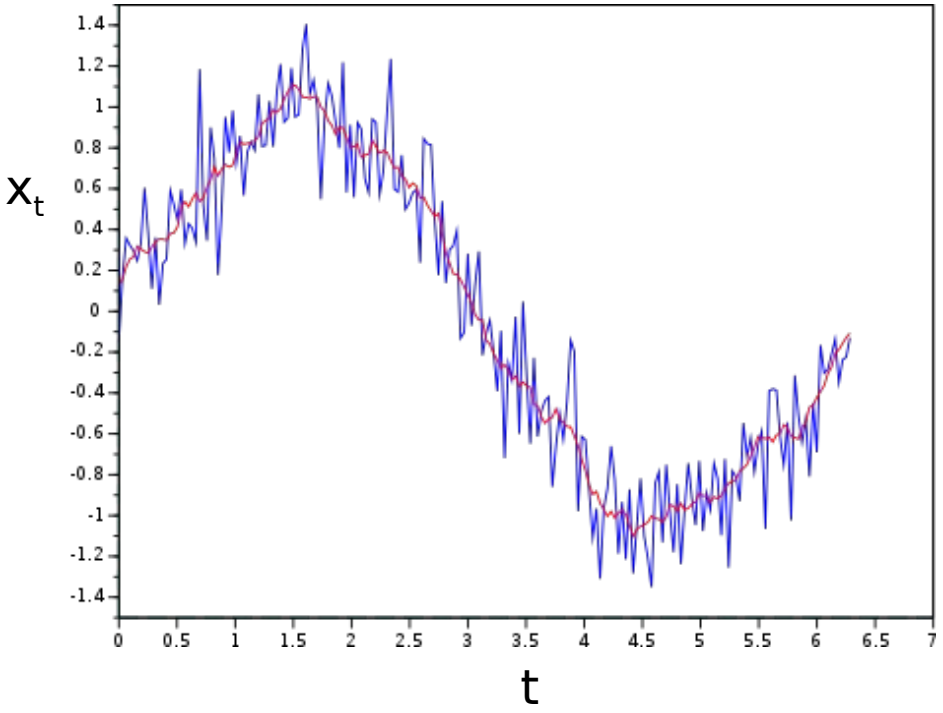
2. Momentum and Nesterov's Accelerated Gradient

The momentum method (Polyak, 1964), which we refer to as classical momentum (CM), is a technique for accelerating gradient descent that accumulates a velocity vector in directions of persistent reduction in the objective across iterations. Given an objective function $f(\theta)$ to be minimized, classical momentum is given by:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

Why it's called Exponential Moving Average?

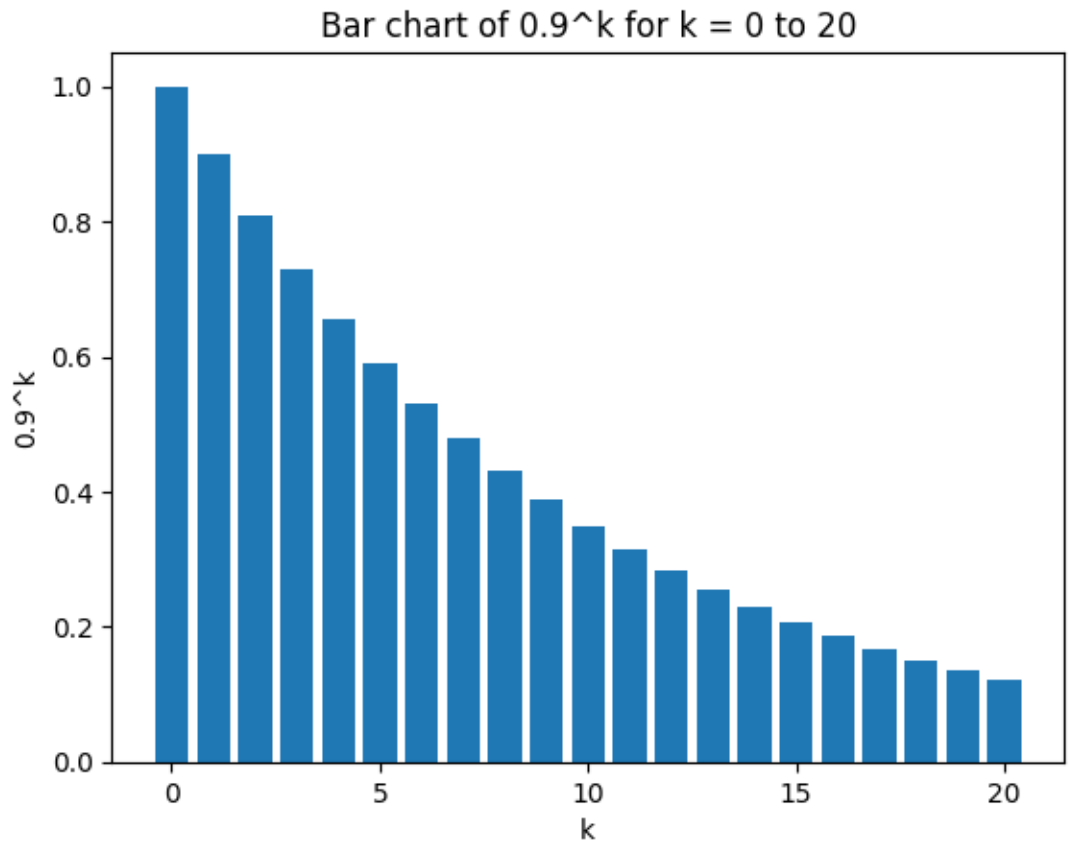


EMA: $$s_t = \alpha x_t + (1-\alpha)s_{t-1}$$

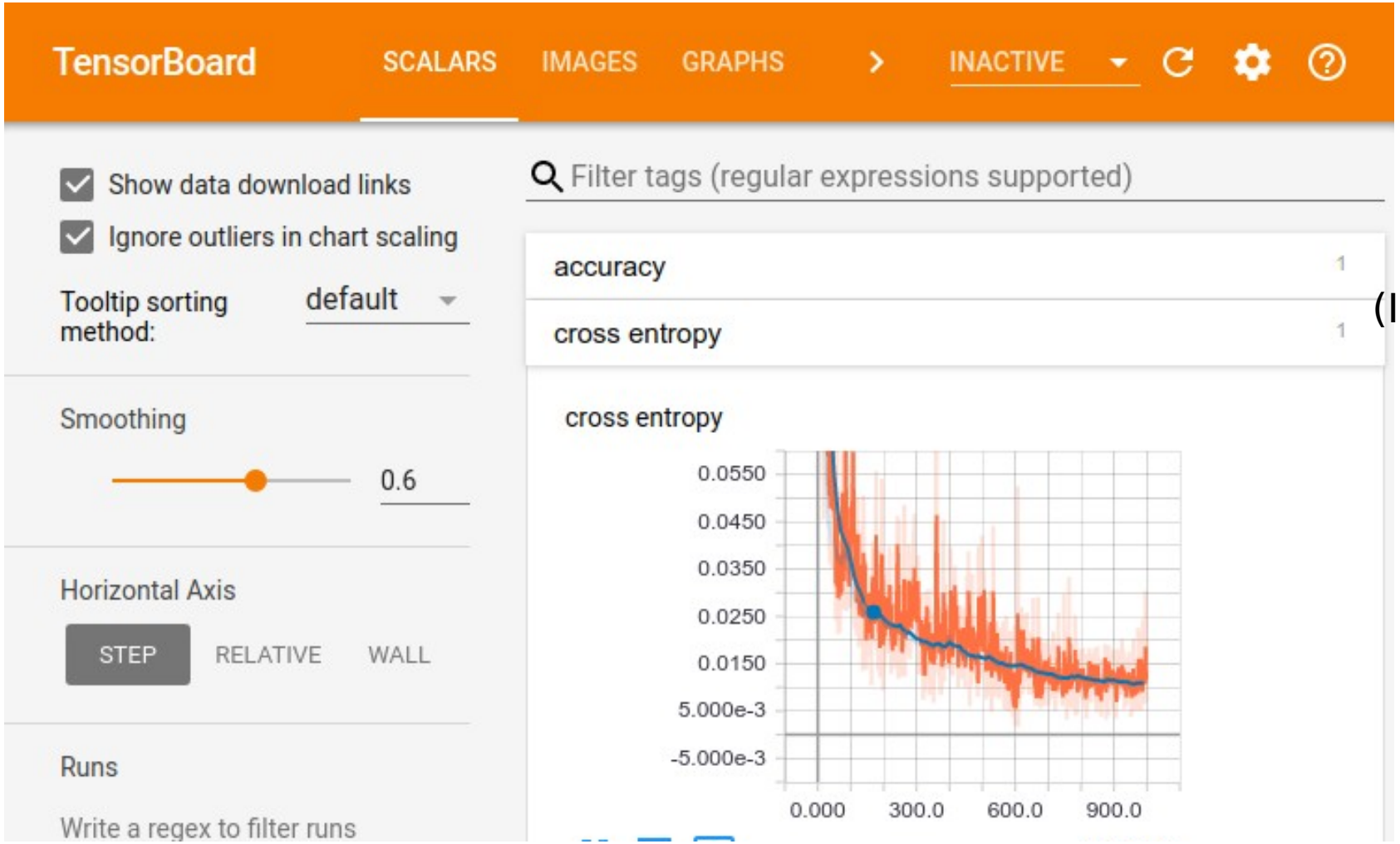
Simple MA: $$S_t = \frac{1}{K}\sum_{k=0}^{K-1} x_{t-k}$$

$$\begin{aligned} s_t &= \alpha x_t + (1-\alpha)s_{t-1} \\ &= \alpha x_t + \alpha(1-\alpha)x_{t-1} + (1-\alpha)^2 s_{t-2} \\ &= \alpha\left[x_t + (1-\alpha)x_{t-1} + (1-\alpha)^2 x_{t-2} + (1-\alpha)^3 x_{t-3} + \cdots + (1-\alpha)^{t-1} x_1\right] + (1-\alpha)^t x_0. \end{aligned}$$

$$
\begin{aligned}
s_t &= \alpha x_t + (1-\alpha) s_{t-1} \\
&= \alpha x_t + \alpha(1-\alpha) x_{t-1} + (1-\alpha)^2 s_{t-2} \\
&= \alpha \left[x_t + (1-\alpha) x_{t-1} + (1-\alpha)^2 x_{t-2} + (1-\alpha)^3 x_{t-3} + \cdots + (1-\alpha)^{t-1} x_1 \right] + (1-\alpha)^t x_0 .
\end{aligned}
$$



Exponential moving averages to smooth out noise



(I guess everyone uses wandb now.

So we if we view the momentum as an EMA of gradients, how could we improve that?

Adam update – exponential moving average

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$$

- Look at typical momentum as EMA of gradients

Adam update – exponential moving average

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2$$

- Look at typical momentum as EMA of gradients
- ADD smoothing of the second moment, uncentered variance of gradients

Adam update – exponential moving average

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon},$$

- Look at typical momentum as EMA of gradients
- ADD smoothing of the second moment, uncentered variance of gradients
- Scale final update accordingly (?)

Adam details

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$
$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$$
$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon},$$

Note: these are *vector* equations – and we have to store **m**, **s**. Unlike SGD, Adam optimizer has a ”state”.

Tiny detail – on early steps, the moving averages will be small (starting at 0), so we bump up the first few updates

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}$$
$$\hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1-\beta_2^t}$$

Adam update – exponential moving average

$$\mathbf{g}_t = \nabla_\theta E(\boldsymbol{\theta})$$
$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$$
$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon},$$

Hyper-parameters:

$$\beta_1 = 0.9$$
$$\beta_2 = 0.999$$
$$\eta = 0.001$$

One of the advantages of Adam is that the defaults work well for many situations. SGD with well-tuned hyper-parameters is usually better, but more effort.

Some variations on Adam

Always use Adam**W** instead of Adam

What if we have **W**eight decay / L2 regularization?

(Show on board if time)

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon},$$

AdamW fixes Adam – doesn't add regularization to gradient, does weight decay of Adam update

If no weight decay, same as Adam.

Memory constraints? AdaFactor, Lion

- For 100B model, storing second moments, $\hat{s}$, is expensive!

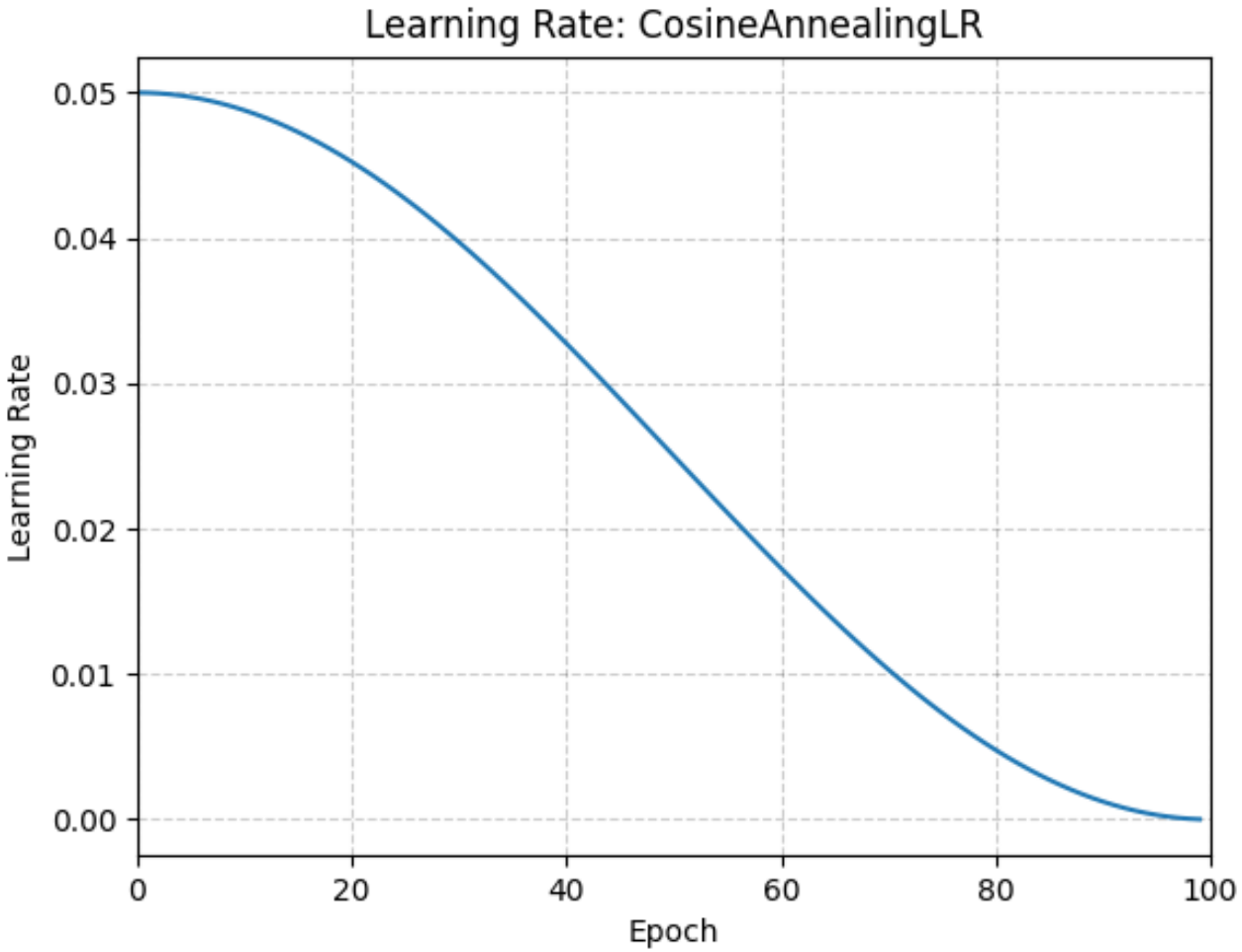
- AdaFactor
  - Store low rank approximation of second moments (for each weight matrix)

- Lion – direction only optimizer
  - Don’t use second moments
  - Update with sign of momentum EMA only (!)

Learning rate schedulers

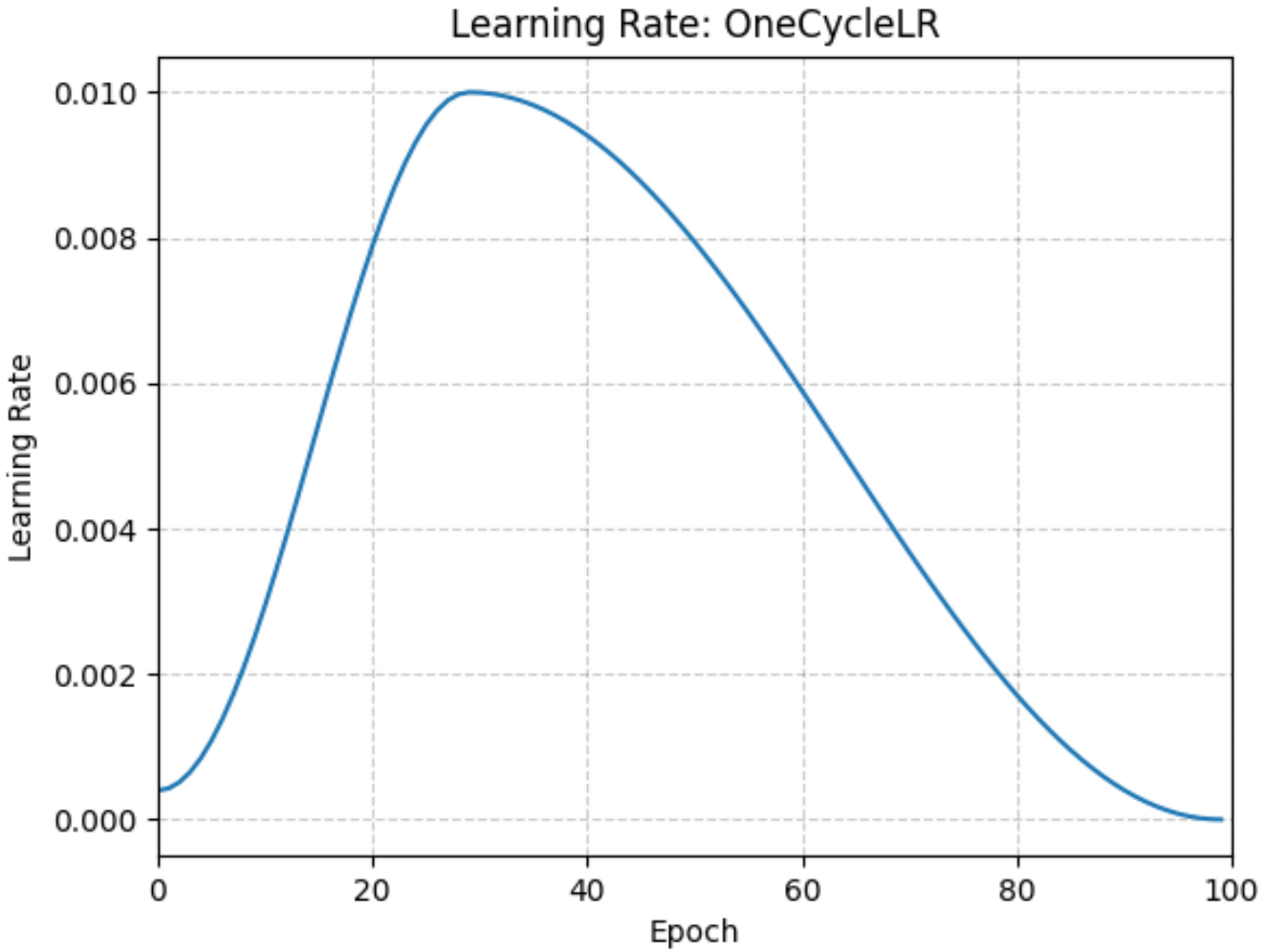
oh great more hyper-parameters to tune

Learning rate decay



Decay helps convergence to good solution

Warmup for more stability

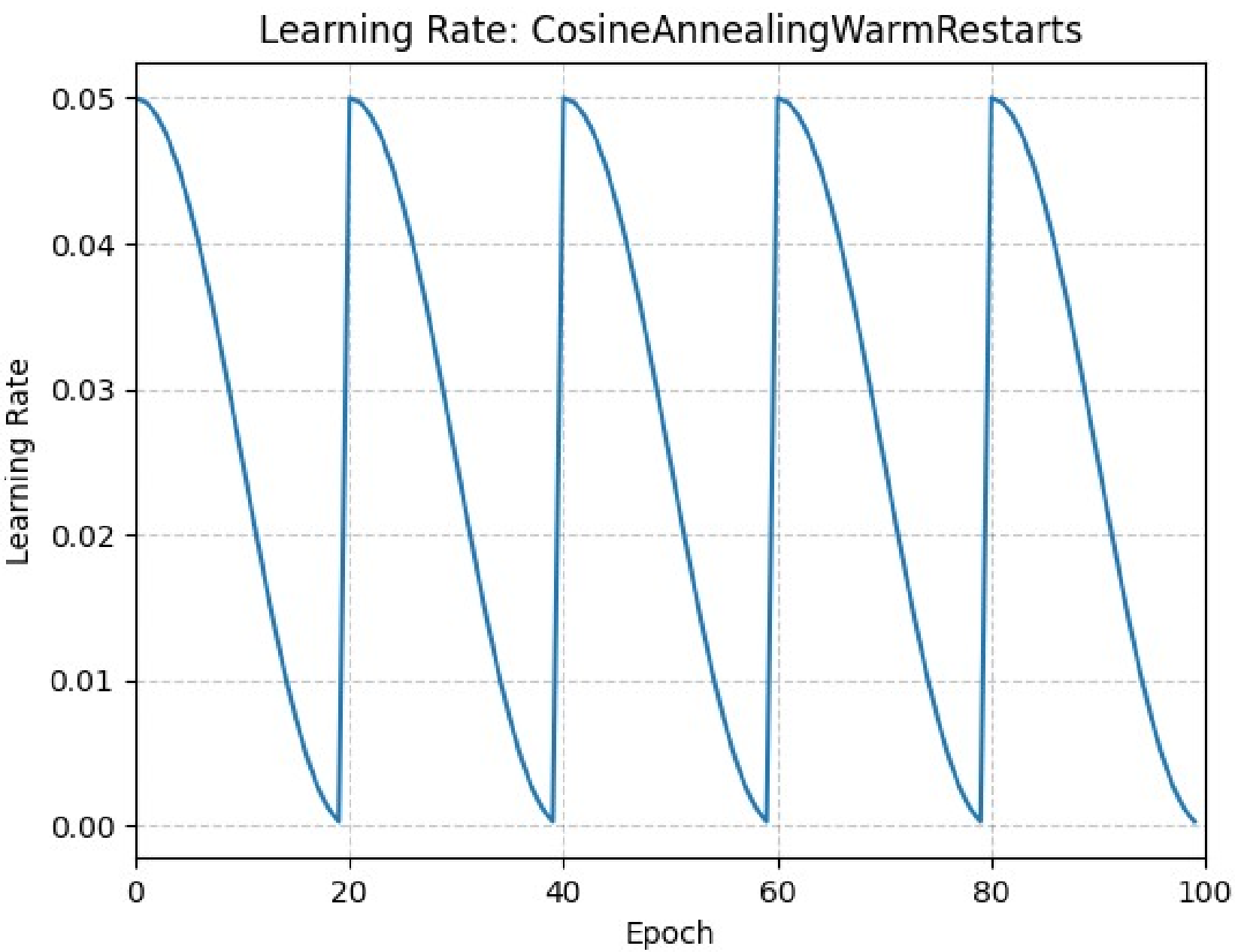


Warmup + decay is typical

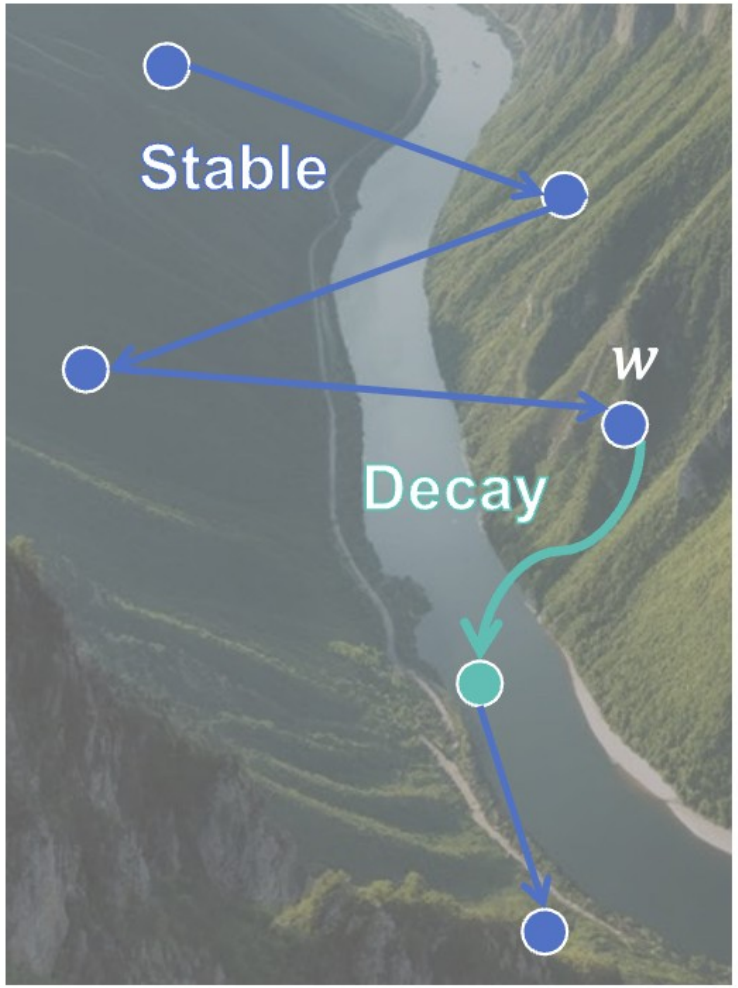
Allows to use a higher LR in the middle

Helps w/ stability for SGD

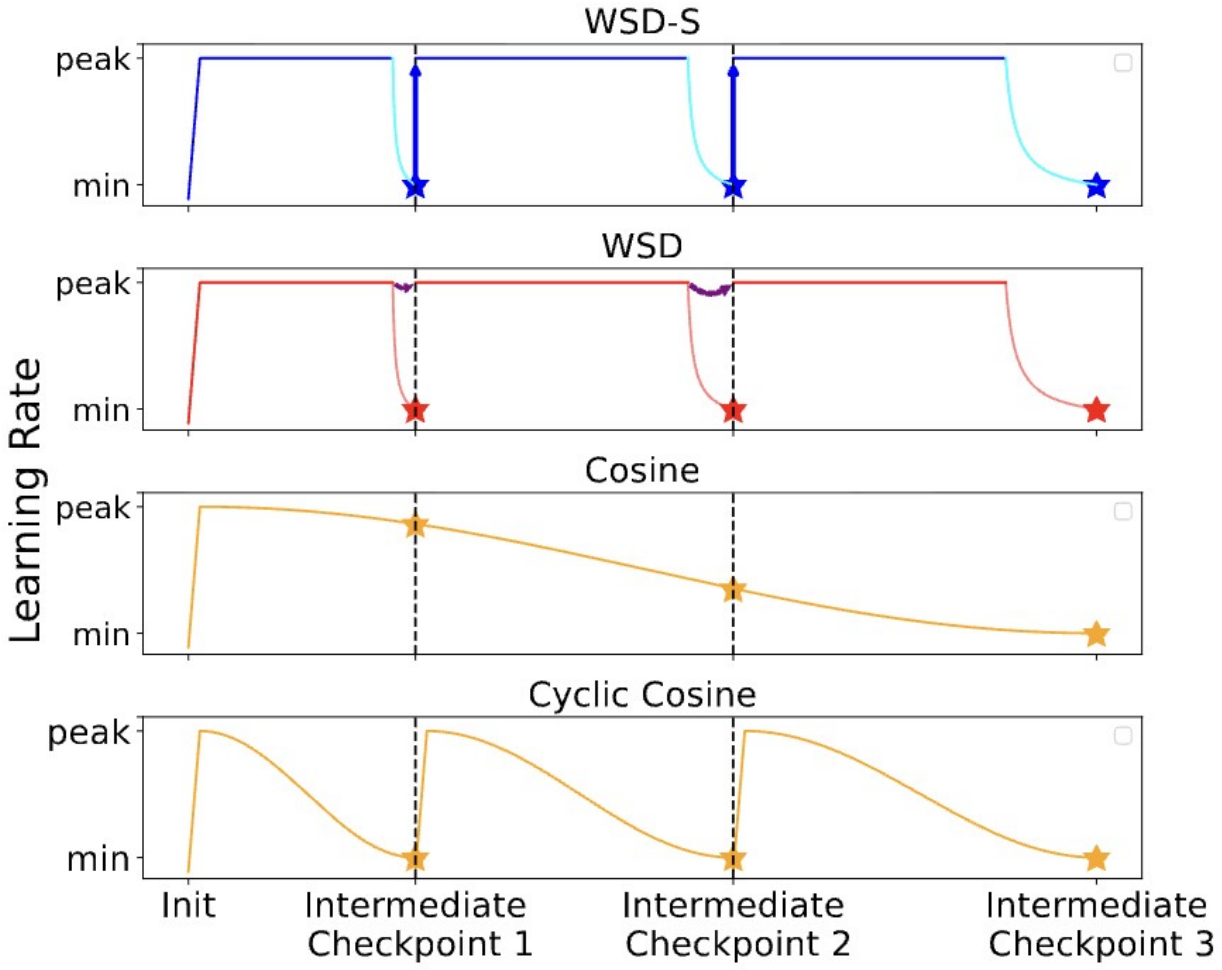
Linear warmup also typical


Learning Rate: CosineAnnealingWarmRestarts
Learning Rate
0.05
0.04
0.03
0.02
0.01
0.00
0
20
40
60
80
100
Epoch

https://arxiv.org/pdf/2410.05192



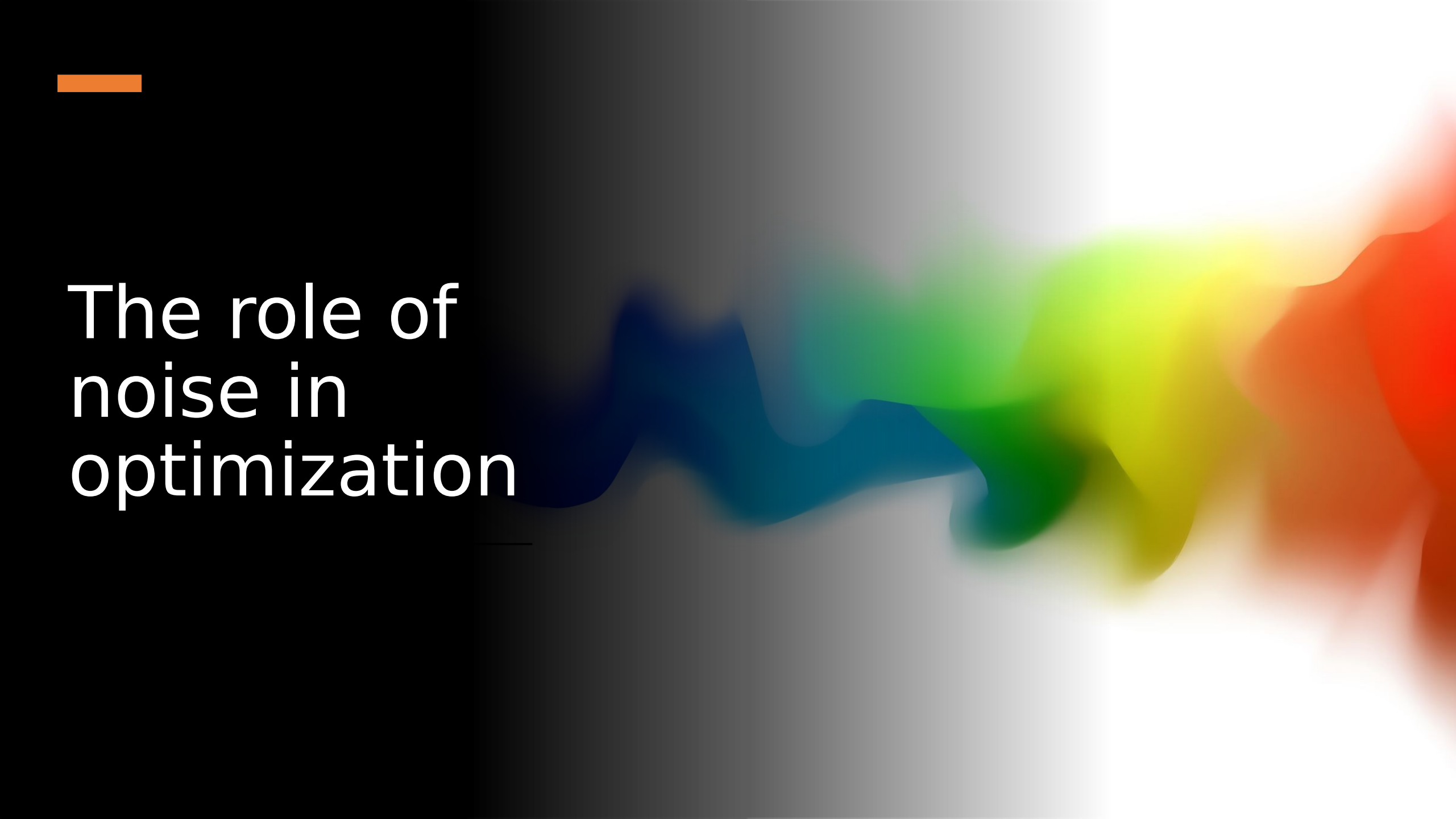
(a) River Valley Landscape



(b) Learning rate schedules

Useful though? Mentioned in

The role of noise in optimization



SGD and why it really works

Stochastic gradient is approximately gradient descent plus normal noise by CLT

$$\frac{\partial L}{\partial w} = \frac{1}{n}\sum_i \partial l(w, x_i)/\partial w = \frac{1}{n}\sum_i g_i = \bar{g}$$

Full dataset loss

True / full dataset gradient

$$g_{minibatch} = \frac{1}{b}\sum_{j \in batch} g_j$$

$$g_{minibatch} \sim N(\bar{g}, \frac{\Sigma}{b})$$

Sum of iid random variables

For large enough batch size, central limit theorem describes distribution

SGD and why it really works

•

What goes here determines the amount of noise or exploration

SGD and why it really works

Continuous limit of this is a stochastic differential equation

$$\Delta x = g(x)\Delta t + s(x)\epsilon$$
$$\epsilon \sim N(0, \Delta t)$$

Compare to our mini-batch update (set covariance to identity):

$$g_{minibatch} \sim N\left(\bar{g}, \frac{1}{b}\right)$$

Could write that as:

$$\Delta x = \bar{g}\,(x)\,\eta + \sqrt{\frac{\eta}{b}}\epsilon$$
$$\epsilon \sim N(0, \eta)$$

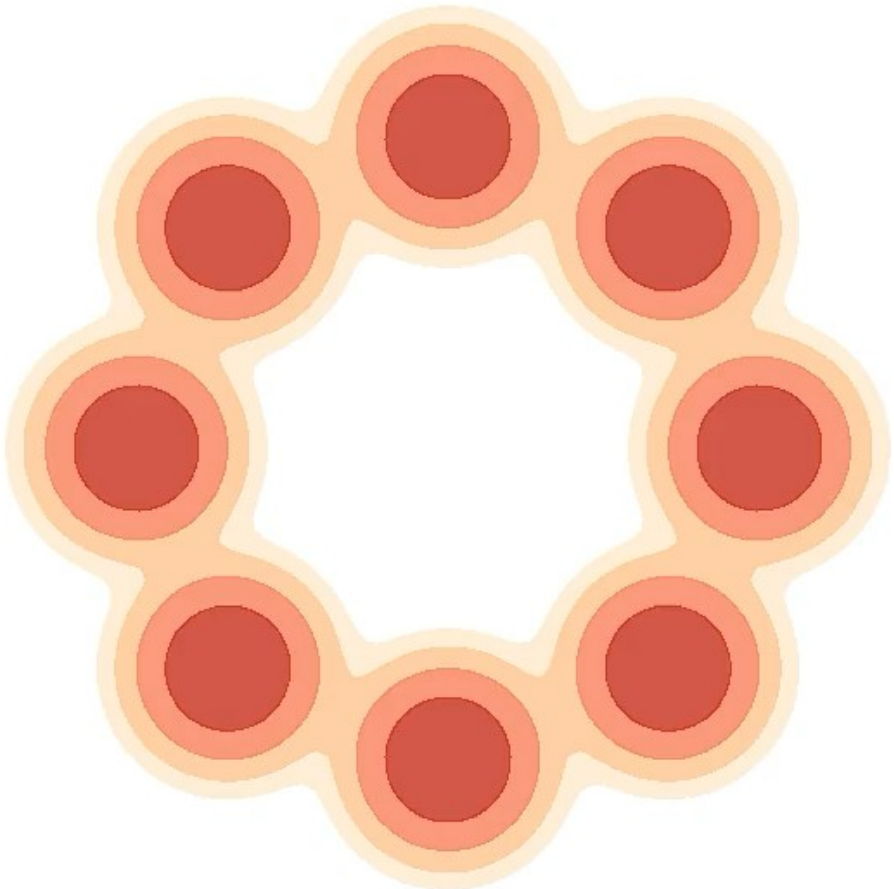
$$\Delta x = \bar{g}\,(x)\,\eta + \sqrt{\frac{\eta}{b}}\epsilon$$
$$\epsilon \sim N(0, \eta)$$

$$\frac{\text{Learning rate}}{\text{Batch size}}$$

Determines the amount of noise or exploration done by SGD

If you change one - change the other!

”Amount of noise / exploration”
What does that really mean?



SGD as stochastic differential equation

1) Stochastic gradient is approximately gradient plus normal noise by CLT

2) Continuous limit is a stochastic differential equation

$$\Delta x = g(x)\Delta t + s(x)\epsilon$$
$$\epsilon \sim N(0, \Delta t)$$

In the limit of small Δt it's written like this ("Ito form" stochastic differential equation):

$$\mathrm{d}x = g(x)\mathrm{d}t + s(x)dW$$

?

Why go to Stochastic Differential Eq. (SDE)?

- Noisy dynamics does not converge to a single point, but rather to a *distribution*

- For SDEs, we can mathematically describe that distribution (thanks to the Fokker-Planck equation)

Fixed points and stationary distributions

Build from fixed points to stationary distributions for discrete case, to stationary distributions for *continuous time and state space...* which we need to understand SGD!!

Fixed points

$x_{t+1} = x_t - \eta\, g(x_t)$ has a fixed point when g(x)=0

Similarly, the continuous version of this dynamics has a fixed point:

$$\dot{x}(t) = -g(x(t))$$

Equivalent for noisy dynamics: stationary distributions

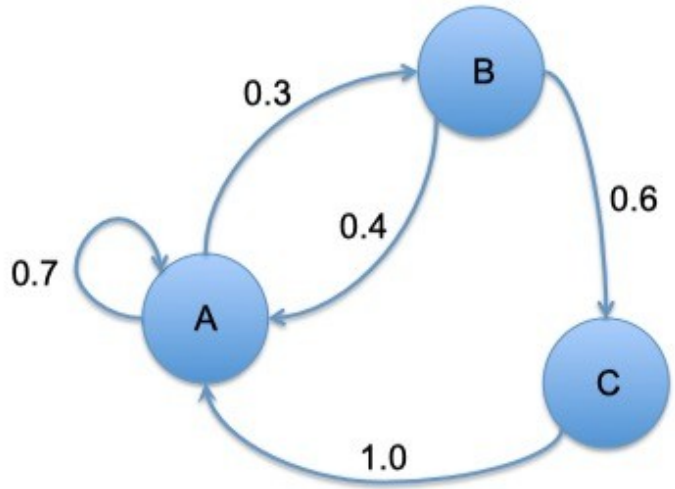
- For *discrete* dynamics, over *discrete* state spaces, we can imagine a Markov chain:

AAABBAAAAABABCAAABABBCAAAAAAABCA…

The transitions between letters are random (and "Markov" – depending on previous state only)

There's no "fixed point" for the state… but the *distribution* has a fixed point (i.e. stationary distribution)

Markov Chains



- Given the current state, the next state is conditionally independent of the history of the process

$$p(x_{t+1}|x_t, x_{t-1}, \cdots, x_0) = p(x_{t+1}|x_t)$$

- Fully characterized by a 3×3 transition matrix,

$$\mathcal{T}_{ij} = Pr(x_{t+1} = i|x_t = j), \;\; i, j \in \{A, B, C\}$$

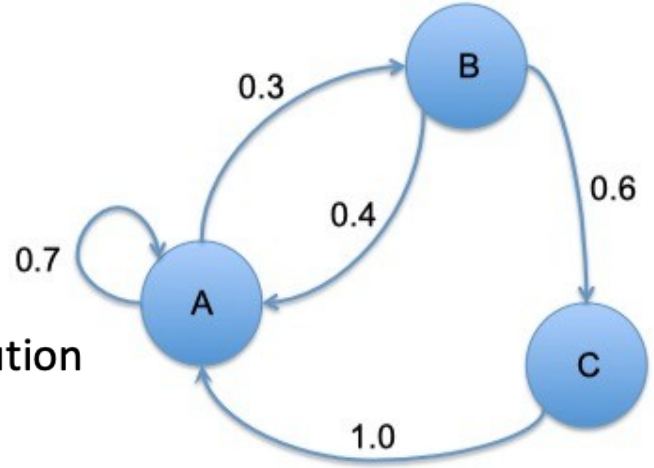
and distribution over initial states, $p_0(x_0 = i), \;\; i \in \{A, B, C\}$

Discrete Markov chains

$$T = \begin{pmatrix} 0.7 & 0.3 & 0 \\ 0.4 & 0 & 0.6 \\ 1 & 0 & 0 \end{pmatrix}$$

"stochastic matrix"
Normalized rows, non-negative
I.e. a conditional probability distribution

$$T(x_t|x_{t-1})$$



$$T^2 = \begin{pmatrix} 0.61 & 0.21 & 0.18 \\ 0.88 & 0.12 & 0. \\ 0.7 & 0.3 & 0. \end{pmatrix}$$

$$T^{20} = \begin{pmatrix} 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \end{pmatrix}$$

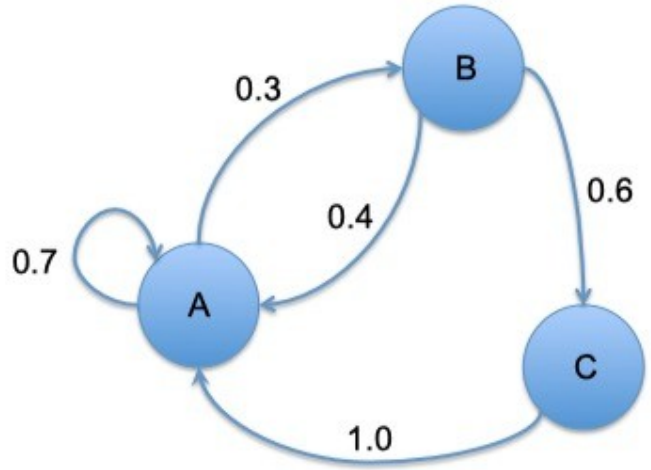
$$T^{50} = \begin{pmatrix} 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \end{pmatrix}$$

$$\boldsymbol{\pi} = \boldsymbol{p_0}\, T^n$$

For large n, any distribution p_0 will lead to the same result

$$T = \begin{pmatrix} 0.7 & 0.3 & 0 \\ 0.4 & 0 & 0.6 \\ 1 & 0 & 0 \end{pmatrix}$$

$$T^{50} = \begin{pmatrix} 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \\ 0.675676 & 0.202703 & 0.121622 \end{pmatrix}$$

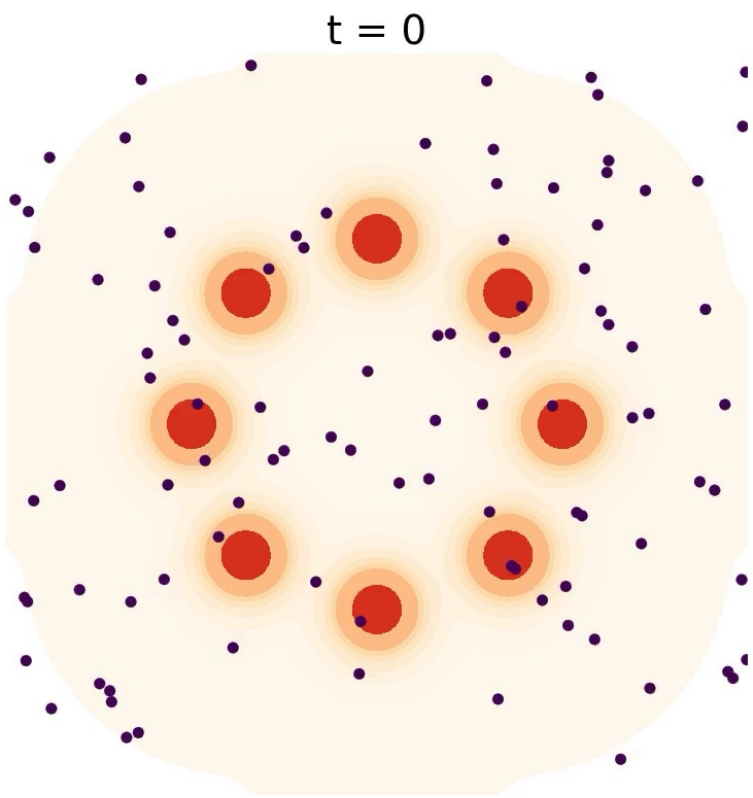


- The eigenvalues of T are $\begin{pmatrix} 1. \\ -0.15 + 0.396863\,i \\ -0.15 - 0.396863\,i \end{pmatrix}$

- And the eigenvector for the largest eigenvalue is $\begin{pmatrix} 0.675676 \\ 0.202703 \\ 0.121622 \end{pmatrix}$

$\boldsymbol{\pi} = \boldsymbol{\pi}\, T$ if $\boldsymbol{\pi} = \begin{pmatrix} 0.675676 \\ 0.202703 \\ 0.121622 \end{pmatrix}$ **I.e., it's a stationary distributio**

Stationary distribution for *continuous* dynamics



- Start with a distribution of random weights
- Simulate with stochastic dynamics

- What is the stationary distribution they converge to? (how to relate *dynamics* and *distribution?*)

Stationary distribution for *continuous* dynamics – **Langevin dynamics**

- Langevin dynamics is defined by this SDE ("overdamped"):

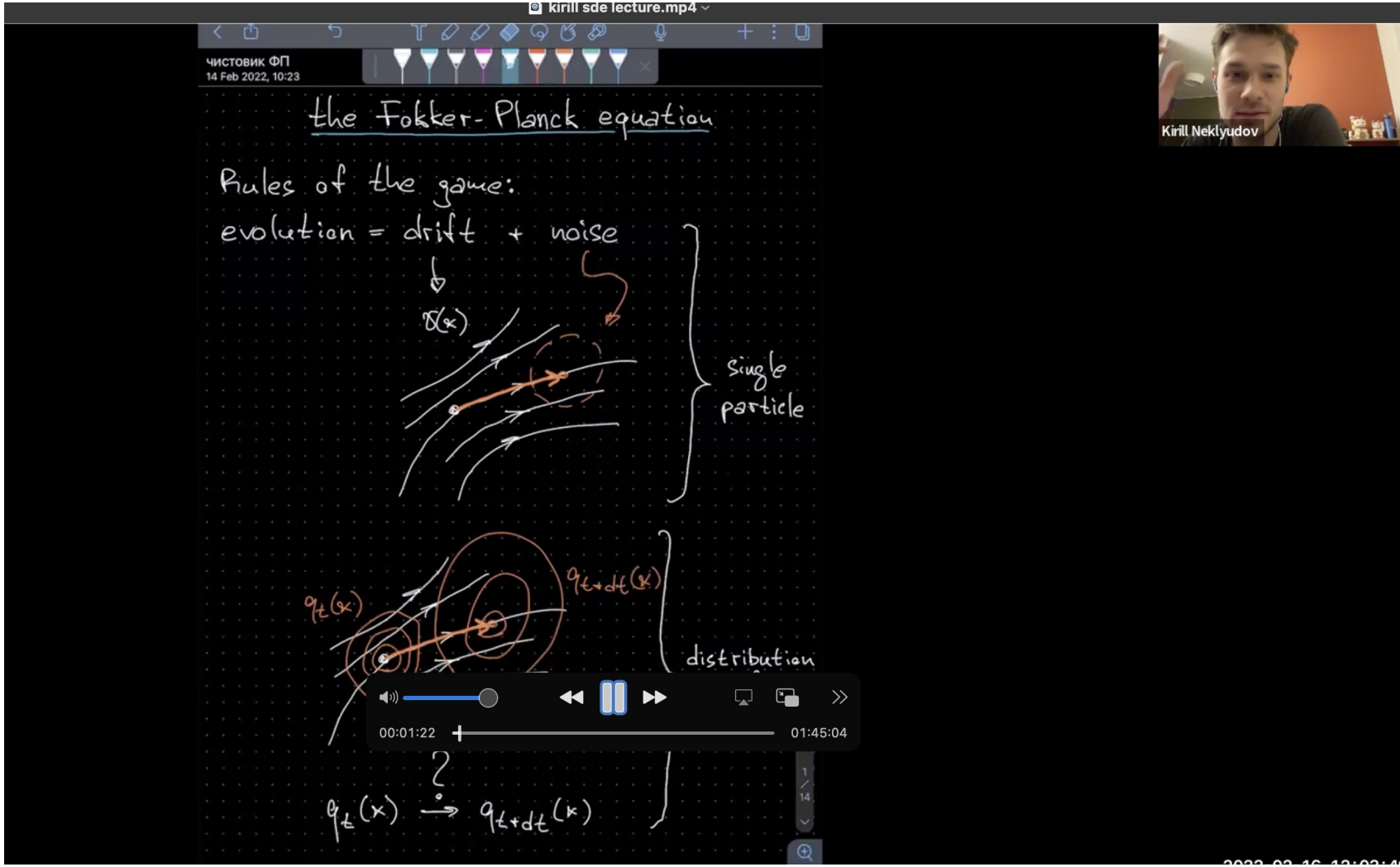
$$dx = -\nabla E(x)dt + \sqrt{2T}dB(t)$$

- The evolution of a probability density on x, under these noisy dynamics is given by the **Fokker Planck** Equation

$$\frac{\partial}{\partial t}p(x,t) = \nabla_{\mathrm{x}} \cdot \left(\nabla_{\mathrm{x}} E(x)\, p(x,t) + \mathrm{T}\, \nabla_{\mathrm{x}} p(x,t)\right)$$

Next level intro to Fokker-Planck – Kirill Neklyudov video

https://www.dropbox.com/scl/fi/tare9lxpu9h1kg43jao4s/kirill-sde-lecture.mp4?rlkey=2smircbw2fuibg9ftj3zdx19r&dl=0

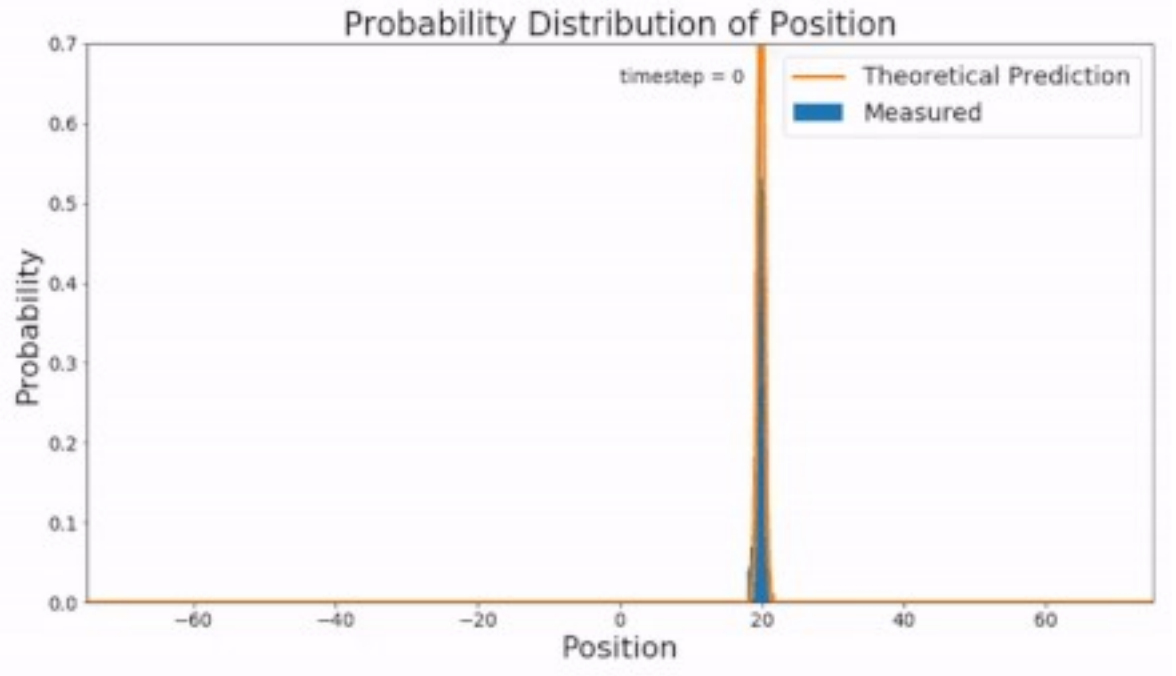


Fokker-Planck is the main useful thing about stochastic SDEs

$$\mathrm{d}X = -\nabla f(X)\,\mathrm{d}t + \sqrt{2\beta^{-1}\,D(X)}\,\mathrm{d}B(t); \quad X(0) = x_0$$

The evolution of a probability density on x,
Under these noisy dynamics is governed by
The Fokker Planck Equation

$$\frac{\partial \rho}{\partial t} = \nabla \cdot \left(\nabla f\, \rho + \beta^{-1}\, \nabla \cdot \left(D\, \rho \right) \right); \quad \rho(0) = \delta(x_0)$$



And even from this, we only really care about stationary distributions, where $\frac{\partial \rho}{\partial t} = 0$

Working out the stationary distribution for Langevin dynamics (Guess and check: Gibbs dist)

$$0 = \nabla \cdot \left(\nabla E(x)\, p(x) + T\, \nabla p(x) \right)$$

Try the solution: $p(x) = e^{-E(x)/T} / Z$

Working out the stationary distribution for Langevin dynamics

Great, we want samples from the distribution,

$$p(x) = e^{-E(x)/T}/Z$$

Which is the stationary distribution of:

$$dx = -\nabla E(x)dt + \sqrt{2T}dB(t)$$

We just need to simulate dynamics!

How? Simulate dynamics by discretizing

Discretization of Langevin

$$dx = -\nabla E(x)dt$$

Euler discretization

$$x_{t+\eta} - x_t = -\nabla E(x)\,\eta$$

SDE (“Ito form”)

$$dx = -\nabla E(x)dt + \sqrt{2T}dW$$

Euler-Maruyama discretization

$$x_{t+\eta} - x_t = -\nabla E(x)\,\eta + \sqrt{2T}\,(W_{t+\eta} - W_t)$$

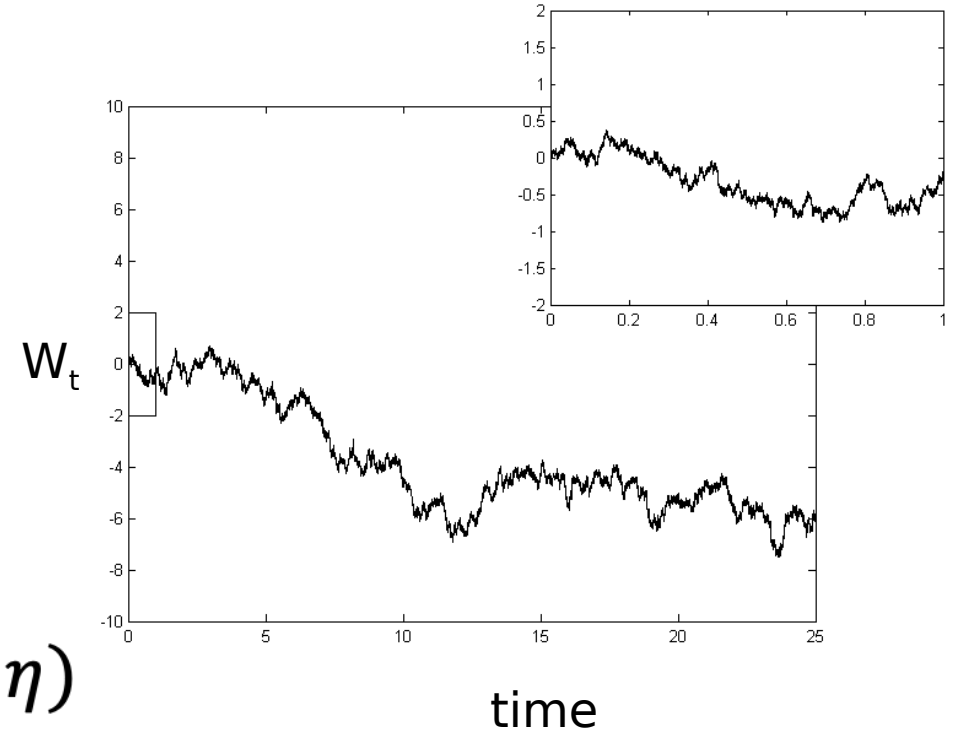
Wiener process

Continuous time random walk process

Or, Brownian motion

For any time t, η:

$$W_{t+\eta} - W_t \sim N(0, \eta)$$

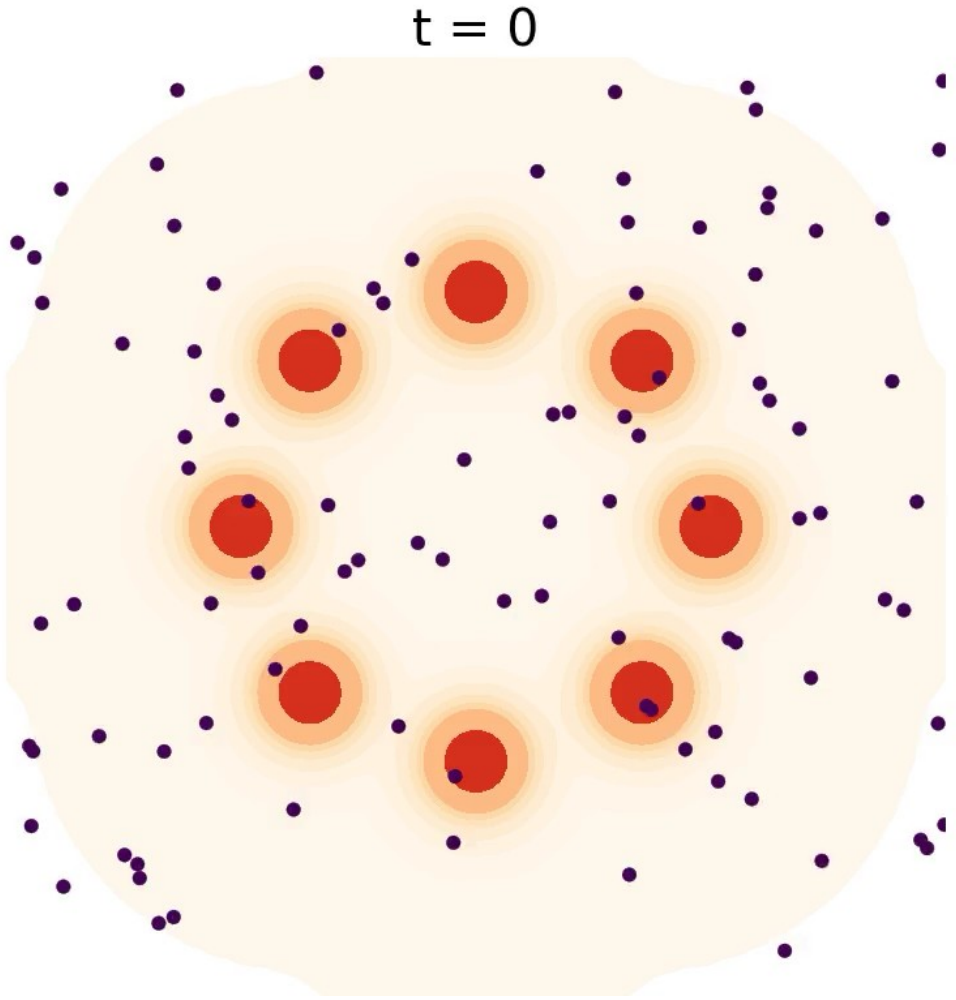


Back to Euler-Maruyama discretization

$$x_{t+\eta} - x_t = -\nabla E(x)\,\eta + \sqrt{2T}\,(W_{t+\eta} - W_t)$$

$$x_{t+\eta} - x_t = -\nabla E(x)\,\eta + \sqrt{2T}\sqrt{\eta}\,v \qquad v \sim N(0, I)$$

Discrete Langevin dynamics*



$$x_{t+\eta} = x_t - \eta \nabla_x E(x_t) + \sqrt{2\eta}\, v$$

Gradient step toward low energy / high probability

Noise to explore $v \sim \mathcal{N}(0, \mathbb{I})$

- Used in many many papers, for a variety of sampling tasks

**Langevin Shenanigans:*

- *Technically, must add Metropolis-Hastings rejection step to make it a valid sampler with discrete steps, or take step size to zero, which no one does*
- *Convergence as t→ ∞, but in practice often t=100 or less (“Non-convergent MCMC”, Nijkamp et al)*

SGD is very similar to Langevin dynamics

$$g_{minibatch} \sim N(\bar{g}, \frac{\Sigma}{b})$$

Let's assume covariance of gradients is Identity (Not true!)

- Because of the noise from minibatches, we found that

$$w_{t+1} = w_t - \eta\, g_{minibatch}$$

$$w_{t+1} = w_t - \eta\, \bar{g} + \frac{\eta}{\sqrt{b}}\, v \qquad v \sim N(0, I)$$

When we discretized a few slides back:

$$w_{t+\eta} - w_t = -\bar{g}\,\eta + \sqrt{2T}\sqrt{\eta}\, v$$

So to get this to match, we need to set the temperature,

$$T = \frac{\eta}{2\, b}$$

SGD is very similar to Langevin dynamics

- So we can approximate SGD as this stochastic differential equation (Langevin dynamics) with a specific temperature T = $\frac{\eta}{2b}$

$$dw = -\nabla E(x)dt + \sqrt{\frac{\eta}{b}}\,dW$$

And Langevin dynamics samples from the stationary distribution,

$$p(x) = e^{-E(x)/T}/Z$$

Take-aways:

- Learning rate and batch size affect exploration/exploitation trade-off
- Learning rate and batch size should be directly related!

Note on approximations.

- The approximation we made, that the gradient covariance is identity (“isotropic”), is not true. See Pratik Chaudhari’s thesis (chapter 6),
- Entropy-SGD: biasing gradient descent into wide valleys,
- Stochastic Gradient Descent Performs Variational Inference…,
- Mandt et al SGD paper

Sometimes people *add* isotropic Gaussian noise to gradient descent, “Stochastic Gradient Langevin Dynamics” (SGLD), to make it more like Langevin dynamics and therefore easier to analyze. From a famous 2011 paper by Welling and Teh

Next week

Monday: Start diffusion